WhiskerFrames - Supplementary Information

Galperin et al.

October 2026 Resubmission

This document serves as supplementary information for the 2026 resubmission of Whisker Frames. It contains an index of supplementary videos, mathematical proofs supporting results in main text sections 2.3 - 2.5, and the source and output of the supporting symbolic math Jupyter Notebook. For all updated source code related to this project, visit the WhiskerFrames Github Repository

Contents

1 Video Index **2**

2 Proofs **2**
2.1 Proof related to Section 2.3: All whisker lines will be tangent to the parabola. 2
2.2 Proof related to Section 2.4: The principal axes of the parabola are spanned by $\boldsymbol{w}$ and $\perp \boldsymbol{w}$. 4
2.3 Proof related to Section 2.4: The focus of the parabola is located at the coordinates (p_1, p_2) in the x-y plane. 5
2.4 Proof related to Section 2.5.3: As θ changes with p_1 and p_2 held constant, the control frame rotates about (p_1, p_2), the focus of the parabola 6

3 Whisker Frames - Symbolic Notebook **8**
3.1 Some helper functions for Equation display 8
3.2 Symbols and setup 8
3.3 (For Section 2.2) A vector model for the mechanism configuration and whisker orientations 9
3.4 (For Section 2.3) Generalizing the analysis to every point in the x-y plane reveals that every control frame configuration is associated with a parabola 10
3.5 (For Section 2.4) A coordinate transformation relates each control frame configuration to the position and orientation of its associated parabola. 13
3.6 (For Section 2.5) The new coordinate system allows for independent control of protraction and fanning 14
3.7 (Proof 4.1.1) Proof that all whisker lines will be tangent to the parabola 16
3.8 (Proof 4.1.2) Proof that the principal axes of the parabola are spanned by w and ⊥w 17
3.9 (Proof 4.1.3) Proof that the focus of the parabola is located at the coordinates (p1,p2) in the x-y plane. 18
3.10 (Proof 4.1.4) Proof that as ϑ changes with p_1and p_2 held constant, the control frame rotates about (p_1, p_2), the focus of the parabola 19
3.11 (For Section 4.3) Hardware Calculations (velocity conversions not in manuscript) . . 20
3.11.1 "forwards" conversions to m and velocity conversions 20
3.11.2 "backwards" conversion from m to r and p 23

1 Video Index

Video 1: Whisker Frames Introduction. In this video, the Whisker Frames project is introduced. A handheld hardware prototype is demonstrated and the biological inspiration of the mechanism is shown.

Video 2: Effects in p-space. This video shows snapshots from Demsos, a browser-based graphing calculator. Isolated effects on the control frame, whisker array, and the output curves (orientation and fanning) are described as variables are moved independently in the (p_1, p_2, θ) coordinate system.

Video 3: Hardware Introduction. This video introduces the robotic hardware discussed in main text sections 2.6 and 4.2.

Video 4: Trajectory Demonstrations. In this video, the robotic hardware is used to demonstrate nine motion trajectories, showing the effects of independent changes in the various coordinate systems described in the main text.

Video 5: Hardware Application Demonstrations. In this video, the robotic hardware demonstrates two extended applications, supporting the uses described in main text section 2.6.

2 Proofs

2.1 Proof related to Section 2.3: All whisker lines will be tangent to the parabola.

In this proof, all whisker lines will be shown to be tangent to the parabola. This will be done by re-deriving the conic equation (5) as an envelope to all possible whisker lines.

First, a whisker line is defined as the line intersecting the point $(0, h)$ and pointing in the direction of the vector $\boldsymbol{u}(h)$. This can be written as a line in standard form:

$$\left(\frac{u_2}{u_1}\right) x - y + h = 0 \tag{32}$$

This equation represents a family of curves in x-y space, as it contains many implicit curves (lines) on the x-y plane defined by a single parameter, h. The family of whisker lines, G, is described as a function of x, y, and h.

$$G(x, y, h) = \left(\frac{u_2}{u_1}\right) x - y + h \tag{33}$$

An envelope to a family of curves is a curve that is tangent to every member of the family. An envelope must satisfy the two conditions at each point it contains:

$$\begin{cases} G(x, y, h) = 0 \\ \frac{\partial G(x,y,h)}{\partial h} = 0 \end{cases} \tag{34}$$

The first condition ensures that the envelope touches G at the point (x, y) for a given value of h. The second condition guarantees that as h varies, G remains tangent to the envelope. The analysis begins by stating the first condition and expanding the equation for the family of whisker lines.

$$G(x, y, h) = \left(\frac{u_2}{u_1}\right) x - y + h = 0 \tag{35}$$

$$\left(\frac{w_2 h - r_2}{w_1 h - r_1}\right) x - y + h = 0 \tag{36}$$

$$(w_2 h - r_2)x - (w_1 h - r_1)y + (w_1 h - r_1)h = 0 \tag{37}$$

Rearranging and grouping by h:

$$G(x, y, h) = (w_1)h^2 + (w_2 x - w_1 y - r_1)h + (-r_2 x + r_1 y) = 0 \tag{38}$$

With an equation established for $G(x, y, h)$, its partial derivative is taken to formulate the second condition.

$$\frac{\partial G(x, y, h)}{\partial h} = w_2 x - w_1 y + 2w_1 h - r_1 = 0 \tag{39}$$

Solving for h:

$$\frac{\partial G(x, y, h)}{\partial h} = 0, \text{ when } h = \frac{-w_2 x + w_1 y + r_1}{2w_1} \tag{40}$$

The envelope conditions are now equivalent to the following system of equations:

$$\begin{cases} (w_2)h^2 + (w_2 x - w_1 y - r_1)h + (-r_2 x + r_1 y) = 0 \\ h = \frac{-w_2 x + w_1 y + r_1}{2w_1} \end{cases} \tag{41}$$

To find the envelope, the system of equations is solved by substituting the second equation into the first:

$$w_2 \left(\frac{-w_2 x + w_1 y + r_1}{2w_1}\right)^2 + (w_2 x - w_1 y - r_1)\left(\frac{-w_2 x + w_1 y + r_1}{2w_1}\right) + (-r_2 x + r_1 y) = 0 \tag{42}$$

Rearranged, (42) becomes

$$\left((w_2 x - w_1 y)^2 - 2r_1(w_2 x - w_1 y) + r_1^2\right) - 4w_1(-r_2 x + r_1 y) = 0 \tag{43}$$

After expansion and reorganization by variable, the resulting expression takes the form of a conic equation:

$$w_2^2x^2 + (-2w_1w_2)xy + w_1^2y^2 + (-2r_1w_2 + 4r_2w_1)x + (-2r_1w_1)y + r_1^2 = 0 \tag{44}$$

This is an identical equation to the one found in section 2.3, proving that the parabola is the envelope to the family of whisker lines and is this tangent to all whisker lines given a configuration of the frames.

2.2 Proof related to Section 2.4: The principal axes of the parabola are spanned by $\boldsymbol{w}$ and $\perp \boldsymbol{w}$.

Recall, the matrix form of the parabolic curve:

$$\boldsymbol{x}^T A \boldsymbol{x}, \text{ where } \boldsymbol{x} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ and } A = \begin{bmatrix} w_2^2 & -w_1w_2 & -r_1w_2 + 2r_2w_1 \\ -w_1w_2 & w_1^2 & -r_1w_1 \\ -r_1w_2 + 2r_2w_1 & -r_1w_1 & r_1^2 \end{bmatrix} \tag{29}$$

By the principal axis theorem, the principal axes of this conic are spanned by the eigenvectors of the 2x2 submatrix

$$A_{sub} = \begin{bmatrix} w_2^2 & -w_1w_2 \\ -w_1w_2 & w_1^2 \end{bmatrix} \tag{45}$$

The eigenvalues of this matrix, λ_1 and λ_1, are determined by solving $\det(A_{sub} - \lambda I) = 0$. Solving for the eigenvalues yields:

$$\det(A_{sub} - \lambda I) = \begin{vmatrix} w_2^2 - \lambda & -w_1w_2 \\ -w_1w_2 & w_1^2 - \lambda \end{vmatrix} = \lambda^2 + (-w_1^2 - w_2^2)\lambda = 0 \tag{46}$$

$$\lambda^2 + (-w_1^2 - w_2^2)\lambda = \lambda\left(\lambda + (-w_1^2 - w_2^2)\right) = 0 \tag{47}$$

$$\lambda_1 = 0,\ \lambda_2 = (w_1^2 + w_2^2) \tag{48}$$

And solving for the eigenvectors using $(A - \lambda I)\boldsymbol{v} = 0$, yields:

$$\text{for } \lambda_1 = 0, \begin{cases} w_2^2 v_1 - w_1w_2v_2 = 0 \\ -w_1w_2v_1 + w_1^2v_2 = 0 \end{cases} \tag{49}$$

$$\text{for } \lambda_2 = (w_1^2 + w_2^2), \begin{cases} -w_1^2 v_1 - w_1w_2v_2 = 0 \\ -w_1w_2v_1 + w_2^2v_2 = 0 \end{cases} \tag{50}$$

$$v_1 = \begin{bmatrix} \frac{w_1}{w_2} \\ 1 \end{bmatrix} c_1 \tag{51}$$

$$v_2 = \begin{bmatrix} \frac{-w_2}{w_1} \\ 1 \end{bmatrix} c_2 \tag{52}$$

where c_1 and c_2 are constants and "$\perp$" indicates perpendicularity. The new basis vectors that define our coordinate transform should lie on the axes spanned by v_1 and v_2, should be of equal length for uniform scaling, and should preserve the handedness of the x-y coordinate system. To satisfy these conditions, the basis vectors are obtained by scaling the eigenvectors by the constant values $c_1 = w_2$ and $c_2 = w_1$, resulting in the following basis vectors:

$$\begin{cases} v_1(w_2) = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \boldsymbol{w} \\ v_2(-w_1) = \begin{bmatrix} w_2 \\ -w_1 \end{bmatrix} = \perp \boldsymbol{w} \end{cases} \tag{53}$$

Thus, the principal axes of the parabola are spanned by the vectors $\boldsymbol{w}$ and $\perp \boldsymbol{w}$, which serve as the foundation for the change-of-basis transformation in (22).

2.3 Proof related to Section 2.4: The focus of the parabola is located at the coordinates (p_1, p_2) in the x-y plane.

This proof establishes that the focus of the parabola derived in section 2.3 is the point (p_1, p_2), when plotted in the x-y plane.

To establish an expression for the focus of the parabola, complex-valued vectors are used to extract information from the conic equation in matrix form. The parabola is defined as the set of all points (x, y) satisfying:

$$\boldsymbol{x}^T A \boldsymbol{x}, \text{ where } \boldsymbol{x} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ and } A = \begin{bmatrix} w_2^2 & -w_1 w_2 & -r_1 w_2 + 2 r_2 w_1 \\ -w_1 w_2 & w_1^2 & -r_1 w_1 \\ -r_1 w_2 + 2 r_2 w_1 & -r_1 w_1 & r_1^2 \end{bmatrix} \tag{29}$$

From projective geometry, all tangent lines to a conic section can be expressed using the inverse of A. We define a line corresponding to the vector $\begin{bmatrix} U & V & W \end{bmatrix}^T$, representing the equation $Ux+Vy+W = 0$. Tangent lines to the parabola satisfy:

$$\begin{bmatrix} U \\ V \\ W \end{bmatrix}^T A^{-1} \begin{bmatrix} U \\ V \\ W \end{bmatrix} = 0, \text{ with the elements of } A^{-1} \text{ labeled as } A^{-1} = \begin{bmatrix} 2G & H & R \\ H & 2K & S \\ R & S & 2T \end{bmatrix} \tag{54}$$

The focus of the parabola is determined using the elements of A^{-1}. Defining the focus as the complex number $F = x_F + i y_F$, corresponding to the point (x_F, y_F) in the x-y plane, the focus of foci of the conic section are found by solving:

$$TF^2 - (R + iS)F + (G - K + iH) = 0 \tag{55}$$

The inverse matrix A^{-1} is then calculated:

$$A^{-1} = \begin{bmatrix} 0 & \frac{-r_1}{2r_1 w_1 w_2 - 2 r_2 w_1^2} & \frac{-1}{2 r_1 w_2 - 2 r_2 w_1} \\ \frac{-r_1}{2r_1 w_1 w_2 - 2 r_2 w_1^2} & \frac{-r_2}{r_1 w_1 w_2 - r_2 w_1^2} & \frac{-w_2}{2r_1 w_1 w_2 - 2 r_2 w_1^2} \\ \frac{-1}{2 r_1 w_2 - 2 r_2 w_1} & \frac{-w_2}{2r_1 w_1 w_2 - 2 r_2 w_1^2} & 0 \end{bmatrix} \tag{56}$$

Because G and T are both zero, the equation for the focus simplifies and rearranges to:

$$F = \frac{-K + iH}{R + iS} \tag{57}$$

Substituting the remaining values, the focus is found to be:

$$F = \frac{-r_2 + ir_1}{w_1 + iw_2} \tag{58}$$

Rearranging to separate real and imaginary components:

$$F = x_F + iy_F = \left(\frac{r_1 w_2 - r_2 w_1}{w_1^2 + w_2^2}\right) + i\left(\frac{r_1 w_1 + r_2 w_2}{w_1^2 + w_2^2}\right) \tag{59}$$

we find that $x_F = p_1$ and $y_F = p_2$, matching equation 24:

$$\boldsymbol{p} = \begin{bmatrix} p_1, p_2 \end{bmatrix}^T = \left[\tfrac{r_1 w_2 - r_2 w_1}{w_1^2 + w_2^2}, \tfrac{r_1 w_1 + r_2 w_2}{w_1^2 + w_2^2}\right]^T \tag{24}$$

Thus, the focus of the parabola will always lie at the coordinate (p_1, p_2).

2.4 Proof related to Section 2.5.3: As θ changes with p_1 and p_2 held constant, the control frame rotates about (p_1, p_2), the focus of the parabola

This proof establishes that when p_1 and p_2 are held constant and θ is changed, the control frame rotates in x-y space with the parabolic focus (p_1, p_2) as the enter of rotation.

As described in the main text, the position and rotation of the control frame is defined by the $[r_1, r_2, \theta]$ coordinate system. The coordinate transformation that expresses the values of r_1 and r_2 in terms of p_1 and p_2 is given by:

$$\boldsymbol{r} = W\boldsymbol{p} = \begin{bmatrix} w_2 p_1 + w_1 p_2 \\ w_2 p_2 - w_1 p_1 \end{bmatrix} \tag{8}$$

To express $\boldsymbol{r}$ as a function of θ, we expand the $\boldsymbol{w}$ terms using the definition in (6). This gives:

$$\boldsymbol{r}(\theta) = \begin{bmatrix} (1 - s\cos\theta)p_1 + (s\sin)p_2 \\ (1 - s\cos\theta)p_2 - (s\sin)p_1 \end{bmatrix} \tag{60}$$

Rearranging:

$$\boldsymbol{r}(\theta) = \begin{bmatrix} sp_2\sin\theta - sp_1\cos\theta + p_1 \\ -sp_1\sin\theta - sp_2\cos\theta + p_2 \end{bmatrix} = s\begin{bmatrix} -p_1 & p_2 \\ -p_2 & -p_1 \end{bmatrix}\begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} + \begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = P\boldsymbol{c}(\theta) + \boldsymbol{p} \tag{61}$$

where

$$P = s\begin{bmatrix} -p_1 & p_2 \\ -p_2 & -p_1 \end{bmatrix}, \text{ and } c(\theta) = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} \tag{62}$$

The rearranged expression for $\boldsymbol{r}(\theta)$ is equal to a transformed circle. The vector $\boldsymbol{c}(\theta)$ encodes the unit circle when plotted for all values of θ. The circle is transformed by the matrix P, and its center is then shifted to lie on the coordinate values of the vector $\boldsymbol{p}$. The resulting shape defines the trajectory of the control frame as θ varies.

Further manipulation of the expression allows P to be rewritten as the product of terms:

$$P = s\begin{bmatrix} -p_1 & p_2 \\ -p_2 & -p_1 \end{bmatrix} = \left(s\sqrt{p_1^2+p_2^2}\right)\begin{bmatrix} \frac{-p_1}{\sqrt{p_1^2+p_2^2}} & \frac{p_2}{\sqrt{p_1^2+p_2^2}} \\ \frac{-p_2}{\sqrt{p_1^2+p_2^2}} & \frac{-p_1}{\sqrt{p_1^2+p_2^2}} \end{bmatrix} = \left(s\sqrt{p_1^2+p_2^2}\right) O \tag{63}$$

where

$$O = \begin{bmatrix} \frac{-p_1}{\sqrt{p_1^2+p_2^2}} & \frac{p_2}{\sqrt{p_1^2+p_2^2}} \\ \frac{-p_2}{\sqrt{p_1^2+p_2^2}} & \frac{-p_1}{\sqrt{p_1^2+p_2^2}} \end{bmatrix} \tag{64}$$

The key property of O is that it is orthogonal, because it satisfies the identity $O^TO = I$. Transformations by an orthogonal matrix preserve both angles and distances, allowing for rotation or reflection without distortion. Notably, the term $\left(s\sqrt{p_1^2+p_2^2}\right)$ is a scalar value. A transformation by an orthogonal matrix multiplied by a scalar is equivalent to an orthogonal transformation followed by a scaling transformation. Therefore, as the unit circle $\boldsymbol{c}(\theta)$ is transformed by the matrix-vector product $P\boldsymbol{c}(\theta)$, the result remains an undistorted circle. The position of the control frame follows the trajectory $\boldsymbol{r}(\theta) = P\boldsymbol{c}(\theta) + \boldsymbol{p}$, which describes a circle centered at $\boldsymbol{p} = (p_1, p_2)$. As θ is changed, the control frame will simultaneously rotate by θ and traverse a circular arc with internal angle θ. Therefore, the control frame rotates about the point (p_1, p_2).

3 Whisker Frames - Symbolic Notebook

2026 - For generating the equations of the Whisker Frames project

[1]:
```
# !pip install sympy
```

[2]:
```
#imports
import sympy as sym
from sympy.solvers import solve
from sympy import symbols, Matrix, sin, cos, tan, cot, atan, simplify, expand,␣
↪Poly, Eq, integrate
```

3.1 Some helper functions for Equation display

[3]:
```
        def multi_Eq(sides):
            if len(sides) == 1:
                raise ValueError('must have at least 2 inputs')
            elif len(sides) == 2:
                return Eq(*sides,evaluate=False)
            elif len(sides) > 2:
                return Eq(sides[0],multi_Eq(sides[1:]),evaluate=False)
            else:
                raise ValueError('not the right input')

        def Eqdisp(*args,label=None,linebreak=True):
            multi_list = [symbols(arg) if type(arg)==str else arg for arg in␣
↪args] #convert any strings to symbols
            if len(multi_list) == 1:
                display_Eq = multi_list[0]
            else:
                display_Eq = multi_Eq(multi_list)
            #display!
            if label is not None:
                print(label)
            display(display_Eq)
            if linebreak:
                print('\n')
```

3.2 Symbols and setup

[4]:
```
        #display variables
        r_d,u_d,w_d,phi_d = symbols('\mathbf{r},\mathbf{u},\mathbf{w},\Phi')
        #configuration variables
        r1,r2,th,u1,u2,w1,w2,a,b,p1,p2 =␣
↪symbols(r'r1,r2,\theta,u1,u2,w1,w2,\alpha,\beta,p1,p2',real=True)
        #parameter and other variables
        s,h,x,y = symbols(r's,h,x,y',real=True)
```

```
lam = symbols(r'\lambda')
```

3.3 (For Section 2.2) A vector model for the mechanism configuration and whisker orientations

[5]:
```
#r vector
r = Matrix([r1,r2])

#"x" configuration
x_conf = Matrix([r1,r2,th])

#define w
w = Matrix([w1,w2])
w_x = Matrix([s*sin(th), 1-s*cos(th)])

#define u as function of r1,r2,theta and as function of r1,r2,w1,w2
u_x = w_x*h - r
u_w = w*h - r
```

[6]:
```
#define protraction in multiple forms
prot_x = atan((u_x[1])/(u_x[0]))
prot_w = atan((u_w[1])/(u_w[0]))
prot_u = atan((u2)/(u1))

#define spread
sprd_x = sym.diff(prot_x,h)
sprd_w = simplify(sym.diff(prot_w,h))
```

[7]:
```
#2.2 equation displays
disps = True
if disps:
    #definitions
    Eqdisp('r',r,label='the vectors r and w',linebreak=False)
    Eqdisp('w',w_x,w)

    #u
    Eqdisp('u',symbols('w')*h - symbols('r'),u_w,label='the vector u is␣
↪defined as a function of r and w')

    #phi and dphi
    Eqdisp(r'\Phi',prot_u,prot_w,label='Phi, or protraction, is defined␣
↪like so')
    Eqdisp(r'\frac{\mathrm{d}\Phi}{\mathrm{d}h}',sprd_w,label='the␣
↪derivative of phi, spread, is defined like so')
```

the vectors r and w

$$r = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}$$

$$w = \begin{bmatrix} s\sin(\theta) \\ -s\cos(\theta) + 1 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

the vector u is defined as a function of r and w

$$u = hw - r = \begin{bmatrix} hw_1 - r_1 \\ hw_2 - r_2 \end{bmatrix}$$

Phi, or protraction, is defined like so

$$\Phi = \operatorname{atan}\left(\frac{u_2}{u_1}\right) = \operatorname{atan}\left(\frac{hw_2 - r_2}{hw_1 - r_1}\right)$$

the derivative of phi, spread, is defined like so

$$\frac{\mathrm{d}\Phi}{\mathrm{d}h} = \frac{-w_1\left(hw_2 - r_2\right) + w_2\left(hw_1 - r_1\right)}{\left(hw_1 - r_1\right)^2 + \left(hw_2 - r_2\right)^2}$$

3.4 (For Section 2.3) Generalizing the analysis to every point in the x-y plane reveals that every control frame configuration is associated with a parabola

[8]:
```
#the arbitrary point will be defined here as z = (x,y)
z = Matrix([x,y])

#q vector
q = Matrix([x,y-h])

#magnitude of cross(u,q)
ucrossq_x = (u_x[0]*q[1])-(u_x[1]*q[0])
ucrossq_w = (u_w[0]*q[1])-(u_w[1]*q[0])

#solve for h when ucrossq = 0
solveh_x = solve(ucrossq_x,h)
solveh_w = solve(ucrossq_w,h)
```

[9]:
```
#get the discriminant (inside square root term) of the solution
sqrt_term_x = solveh_x[0].args[3].args[2].args[1]
sqrt_term_w = -solveh_w[0].args[2].args[1]
```

```
        #the conic equation is equal to the descriminant
        conic_x = sqrt_term_x**2
        conic_w = sqrt_term_w**2

        #wrap the conic in a Polynomial object
        conic_poly_x = Poly(conic_x, [x,y])
        conic_poly_w = Poly(conic_w, [x,y])

        #Get coefficients
        coeffs_x = conic_poly_x.coeffs()
        coeffs_w = conic_poly_w.coeffs()
```

[10]:
```
        #piece together the matrix form
        a,b,d,c,e,f = coeffs_x
        A_x = Matrix([[a,b/2,d/2],
                      [b/2,c,e/2],
                      [d/2,e/2,f]])
        a,b,d,c,e,f = coeffs_w
        A_w = Matrix([[a,b/2,d/2],
                      [b/2,c,e/2],
                      [d/2,e/2,f]])

        #classification
        A33 = A_w[0:2,0:2]
        detA33 = A33.det()

        #degenerate conditions
        degeneracy = A_w.det().simplify()
```

[11]:
```
        #2.3 equation displays
        disps = True
        if disps:
            #setup
            Eqdisp('q',q,label='the vector q')
            Eqdisp(ucrossq_w,label='the value of |u x q|')

            #solving for the parabola
            Eqdisp('h',solveh_w[0],label='solving for h when u x q = 0')
            Eqdisp('conic',conic_w,label='value of the discriminant is a conic␣
↪section (parabola)')

            #matrix form and degeneracy
            Eqdisp('conic',r'{z^T}Az',label='we can express the parabola in␣
↪matrix form',linebreak=False)
            Eqdisp('A',A_w,linebreak=False)
            Eqdisp('z',Matrix([x,y,1]))
```

```
Eqdisp(degeneracy,0,label='the parabola is degenerate when the following expression is satisfied')
```

the vector q

$$q = \begin{bmatrix} x \\ -h + y \end{bmatrix}$$

the value of |u x q|

$$-x\left(hw_2 - r_2\right) + \left(-h + y\right)\left(hw_1 - r_1\right)$$

solving for h when u x q = 0

$$h = \frac{r_1 + w_1 y - w_2 x - \sqrt{r_1^2 - 2r_1 w_1 y - 2r_1 w_2 x + 4r_2 w_1 x + w_1^2 y^2 - 2w_1 w_2 xy + w_2^2 x^2}}{2w_1}$$

value of the discriminant is a conic section (parabola)

$$conic = r_1^2 - 2r_1 w_1 y - 2r_1 w_2 x + 4r_2 w_1 x + w_1^2 y^2 - 2w_1 w_2 xy + w_2^2 x^2$$

we can express the parabola in matrix form

$$conic = z^T A z$$

$$A = \begin{bmatrix} w_2^2 & -w_1 w_2 & -r_1 w_2 + 2r_2 w_1 \\ -w_1 w_2 & w_1^2 & -r_1 w_1 \\ -r_1 w_2 + 2r_2 w_1 & -r_1 w_1 & r_1^2 \end{bmatrix}$$

$$z = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

the parabola is degenerate when the following expression is satisfied

$$4w_1^2\left(-r_1^2 w_2^2 + 2r_1 r_2 w_1 w_2 - r_2^2 w_1^2\right) = 0$$

3.5 (For Section 2.4) A coordinate transformation relates each control frame configuration to the position and orientation of its associated parabola.

[12]:
```
        #find the axes of the parabola via principal axis theorem (only doing␣
↪this for A_w)
        A33 = A_w[0:2,0:2]
        A_ev = A33.eigenvects()
        evec1,evec2 = A_ev[0][2][0], A_ev[1][2][0]

        #define change-of-basis matrix W
        W = Matrix([[w2,w1],[-w1,w2]])
        W_inv = W.inv()

        #define p vector
        p_p = Matrix([p1,p2])

        #get p to r conversions
        r_p = W*p_p
        p_r = W_inv*r
        p_x = simplify(p_r.subs({w1:w_x[0],w2:w_x[1]}))

        #new configuration
        x_3_3 = Matrix([p1,p2,th])
```

[13]:
```
        #2.4 displays
        Eqdisp(r'A_{33}',A33,label='The 2x2 submatrix')
        Eqdisp(r'v_{1}',evec1,label='Calculate the eigenvectors of A33: v1 and␣
↪v2',linebreak=False)
        Eqdisp(r'v_{2}',evec2)
        Eqdisp('W',W,label='The change of basis matrix')
        Eqdisp('r','Wp',r_p,label='r in terms of p')
        Eqdisp('p',r'W^{-1}r',p_r,'parabolicfocus',label='p in terms of r')
        Eqdisp('p',p_x,label='p in terms of r, s, theta')
```

The 2x2 submatrix

$$A_{33} = \begin{bmatrix} w_2^2 & -w_1 w_2 \\ -w_1 w_2 & w_1^2 \end{bmatrix}$$

Calculate the eigenvectors of A33: v1 and v2

$$v_1 = \begin{bmatrix} \frac{w_1}{w_2} \\ 1 \end{bmatrix}$$

$$v_2 = \begin{bmatrix} -\frac{w_2}{w_1} \\ 1 \end{bmatrix}$$

The change of basis matrix

$$W = \begin{bmatrix} w_2 & w_1 \\ -w_1 & w_2 \end{bmatrix}$$

r in terms of p

$$r = Wp = \begin{bmatrix} p_1 w_2 + p_2 w_1 \\ -p_1 w_1 + p_2 w_2 \end{bmatrix}$$

p in terms of r

$$p = W^{-1} r = \begin{bmatrix} \frac{r_1 w_2}{w_1^2+w_2^2} - \frac{r_2 w_1}{w_1^2+w_2^2} \\ \frac{r_1 w_1}{w_1^2+w_2^2} + \frac{r_2 w_2}{w_1^2+w_2^2} \end{bmatrix} = parabolic focus$$

p in terms of r, s, theta

$$p = \begin{bmatrix} \frac{-r_1 s\cos(\theta)+r_1-r_2 s\sin(\theta)}{s^2-2s\cos(\theta)+1} \\ \frac{r_1 s\sin(\theta)-r_2 s\cos(\theta)+r_2}{s^2-2s\cos(\theta)+1} \end{bmatrix}$$

3.6 (For Section 2.5) The new coordinate system allows for independent control of protraction and fanning

[14]:
```
#get u in terms of p. Simplify using U matrix
u_p = w*h-r_p
U = Matrix([[h-p2,-p1],[p1,h-p2]])
# simplify(u_p[0]**2 + u_p[1]**2)

#calculate protraction and spread
prot_p = atan((u_p[1])/(u_p[0]))
sprd_p = simplify(sym.diff(prot_p,h))

#more justification for sprd_p
sprd_num = expand(u_p[0]**2)
sprd_denom = expand(u_p[0]**2 + u_p[1]**2)
# Eqdisp('num',sprd_num)
# Eqdisp('denom',sprd_denom)

#differentiating phi with respect to theta...
dphi_dth = simplify(prot_p.subs({w1:w_x[0],w2:w_x[1]}).diff(th))
```

```
        #verifying the expression in (31) is a correct antiderivative (this␣
↪expression was obtained by hand, verified here)
        soln_31 = th-sym.atan((s-cos(th))/(sin(th)))
        is_d31_30 = dphi_dth.equals(soln_31.diff(th))
```

[15]:
```
        #2.5 displays
        Eqdisp('u',symbols('w')*h-symbols('Wp'),u_p,'Uw',label='the u vector in␣
↪terms of p',linebreak=False)
        Eqdisp('U',U)
        Eqdisp(symbols('u')-symbols('Uw'),simplify(u_p-U*w),label='check that␣
↪matrix U is correctly defined')
        Eqdisp('Phi',prot_p,label='protraction with respect to p')
        Eqdisp(r'\frac{\mathrm{d}\Phi}{\mathrm{d}h}',sprd_p,label='spread with␣
↪respect to p')
        Eqdisp('(31)',soln_31,label="equation 31 is the antiderivative of dphi/
↪dth. (hard-coded here, obtained on paper)")
        Eqdisp(is_d31_30,label="is the expression in (31) a correct␣
↪antiderivative of dphi/dth?")
```

the u vector in terms of p

$$u = -Wp + hw = \begin{bmatrix} hw_1 - p_1 w_2 - p_2 w_1 \\ hw_2 + p_1 w_1 - p_2 w_2 \end{bmatrix} = Uw$$

$$U = \begin{bmatrix} h - p_2 & -p_1 \\ p_1 & h - p_2 \end{bmatrix}$$

check that matrix U is correctly defined

$$-Uw + u = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

protraction with respect to p

$$\Phi = \operatorname{atan}\left(\frac{hw_2 + p_1 w_1 - p_2 w_2}{hw_1 - p_1 w_2 - p_2 w_1}\right)$$

spread with respect to p

$$\frac{\mathrm{d}\Phi}{\mathrm{d}h} = -\frac{p_1}{h^2 - 2hp_2 + p_1^2 + p_2^2}$$

```
        equation 31 is the antiderivative of dphi/dth. (hard-coded here, obtained␣
↪on
        paper)
```

$$(31) = \theta - \operatorname{atan}\left(\frac{s - \cos(\theta)}{\sin(\theta)}\right)$$

```
        is the expression in (31) a correct antiderivative of dphi/dth?
        True
```

3.7 (Proof 4.1.1) Proof that all whisker lines will be tangent to the parabola

The envelope conditions are...

$$\begin{cases} G(x,y,t) = 0 \\ \frac{\partial F}{\partial t} = 0 \end{cases}$$

Where G is a family of curves, here specifically its the whisker lines. h is the single parameter that defines the family of lines.

[16]:
```
        #define G(x,y,h)
        G = (u_w[1]/u_w[0])*x - y + h

        #now multiply it by u2 since you can do that in the equation equal to␣
↪zero...
        G_mult = sym.collect(sym.expand(sym.simplify(G*u_w[0])),h)

        #take the derivative of G_mult with respect to h to form the second␣
↪envelope condition
        condition2 = G_mult.diff(h)

        #solve for h
        h_solve = solve(condition2,h)[0]

        #solve system of eqns by substituting h into G=0
        conditions_solve = sym.simplify(G_mult.subs({h:h_solve}))

        #again, since its equal to zero, we can multiply ("both sides") by (-4w1)
        conditions_solve_mult = conditions_solve*(-4*w1)

        # is it equal to the conic?
        is_equal_to_conic = conditions_solve_mult.equals(conic_w)
```

[17]:
```
#4.1 displays
Eqdisp("G",(u2/u1)*x-y+h,G,label='The family of curves G(x,y,h)')
Eqdisp(r'\frac{\mathrm{d}G}{\mathrm{d}h}',0,condition2,label='Derivative␣
↪of G with respect to h (second envelope condition)')
Eqdisp('h_G',h_solve, label='first condition (G=0) solved for h')
Eqdisp(conditions_solve_mult,0,label = 'solution to system of equations')
Eqdisp(is_equal_to_conic,label='is it equal to the conic?')
```

The family of curves G(x,y,h)

$$G = h - y + \frac{u_2 x}{u_1} = h + \frac{x\left(hw_2 - r_2\right)}{hw_1 - r_1} - y$$

Derivative of G with respect to h (second envelope condition)

$$\frac{\mathrm{d}G}{\mathrm{d}h} = 0 = 2hw_1 - r_1 - w_1 y + w_2 x$$

first condition (G=0) solved for h

$$h_G = \frac{r_1 + w_1 y - w_2 x}{2w_1}$$

solution to system of equations

$$-4w_1\left(r_1 y - r_2 x - \frac{\left(r_1 + w_1 y - w_2 x\right)^2}{4w_1}\right) = 0$$

is it equal to the conic?

True

3.8 (Proof 4.1.2) Proof that the principal axes of the parabola are spanned by w and $\perp$w

[18]:
```
# take the eigenvalues of the A33 () matrix...
v1 = A33.eigenvects()[0][2][0]
v2 = A33.eigenvects()[1][2][0]

#multiply by chosen constants
basis_1 = v1*w2
```

```
basis_2 = v2*w1
```

[19]:
```
#4.2 displays
Eqdisp('v_1',v1,label='eigenvalues of A33')
Eqdisp('v_2',v2)
Eqdisp(r'\mathrm{basis_1}',basis_1,label='chosen basis vectors')
Eqdisp(r'\mathrm{basis_2}',basis_2)
```

eigenvalues of A33

$$v_1 = \begin{bmatrix} \frac{w_1}{w_2} \\ 1 \end{bmatrix}$$

$$v_2 = \begin{bmatrix} -\frac{w_2}{w_1} \\ 1 \end{bmatrix}$$

chosen basis vectors

$$\mathrm{basis}_1 = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

$$\mathrm{basis}_2 = \begin{bmatrix} -w_2 \\ w_1 \end{bmatrix}$$

3.9 (Proof 4.1.3) Proof that the focus of the parabola is located at the coordinates (p1,p2) in the x-y plane.

[20]:
```
#invert A
A_w_inv = A_w.inv()

#get values...
K = A_w_inv[1,1]/2
H = A_w_inv[0,1]
S = A_w_inv[1,2]
R = A_w_inv[0,2]

#calculate focus...
focus = sym.simplify((-K + H*sym.I)/(R + S*sym.I))

#get real and imaginary parts
```

```
        xF = sym.re(focus)
        yF = sym.im(focus)
```

[21]:
```
        #4.3 displays
        Eqdisp('A^-1',A_w_inv,label='Inverse of A')
        Eqdisp('F',focus,label='focus of the parabola')
        Eqdisp('x_F','p_1',xF,label='The real and imaginary parts of this focus␣
↪are equal to the coordinates of the point (p1,p2)')
        Eqdisp('x_F','p_2',yF)
```

Inverse of A

$$A^{-1} = \begin{bmatrix} 0 & -\frac{r_1}{2r_1w_1w_2-2r_2w_1^2} & -\frac{1}{2r_1w_2-2r_2w_1} \\ -\frac{r_1}{2r_1w_1w_2-2r_2w_1^2} & -\frac{r_2}{r_1w_1w_2-r_2w_1^2} & -\frac{w_2}{2r_1w_1w_2-2r_2w_1^2} \\ -\frac{1}{2r_1w_2-2r_2w_1} & -\frac{w_2}{2r_1w_1w_2-2r_2w_1^2} & 0 \end{bmatrix}$$

focus of the parabola

$$F = \frac{ir_1 - r_2}{w_1 + iw_2}$$

The real and imaginary parts of this focus are equal to the coordinates of␣
↪the
point (p1,p2)

$$x_F = p_1 = \frac{r_1w_2}{w_1^2+w_2^2} - \frac{r_2w_1}{w_1^2+w_2^2}$$

$$x_F = p_2 = \frac{r_1w_1}{w_1^2+w_2^2} + \frac{r_2w_2}{w_1^2+w_2^2}$$

3.10 (Proof 4.1.4) Proof that as ϑ changes with p_1and p_2 held constant, the control frame rotates about (p_1, p_2), the focus of the parabola

[22]:
```
        #substitute s and theta terms into the p-space expression for r
        r_p_subs = r_p.subs({w1:w_x[0],w2:w_x[1]})

        #verify that the O matrix, derived from the P matrix (see paper), is␣
↪orthogonal
        P = Matrix([[-p1,p2],[-p2,-p1]])
        O = sym.simplify(P/(sym.sqrt(p1**2 + p2**2)))
```

```
        is_equal_to_I = (O.T*O).equals(sym.eye(2))
```

[23]:
```
        Eqdisp('r',r_p_subs,label='r, in p space, expressed in s and theta␣
↪terms')
        Eqdisp('O',O,label='O matrix')
        Eqdisp(is_equal_to_I,label='is O orthogonal?')
```

r, in p space, expressed in s and theta terms

$$r = \begin{bmatrix} p_1 \left(-s\cos\left(\theta\right)+1\right)+p_2 s\sin\left(\theta\right) \\ -p_1 s\sin\left(\theta\right)+p_2\left(-s\cos\left(\theta\right)+1\right) \end{bmatrix}$$

O matrix

$$O = \begin{bmatrix} -\frac{p_1}{\sqrt{p_1^2+p_2^2}} & \frac{p_2}{\sqrt{p_1^2+p_2^2}} \\ -\frac{p_2}{\sqrt{p_1^2+p_2^2}} & -\frac{p_1}{\sqrt{p_1^2+p_2^2}} \end{bmatrix}$$

is O orthogonal?

True

3.11 (For Section 4.3) Hardware Calculations (velocity conversions not in manuscript)

3.11.1 "forwards" conversions to m and velocity conversions

[24]:
```
        #some symbols
        delta1,delta2,m1,m2 = symbols(r"\Delta_1,\Delta_2,m_1,m_2")
        m = Matrix([m1,m2])
        delta = Matrix([delta1,delta2])
        Rot = Matrix([[cos(th),-sin(th)],[sin(th), cos(th)]])
        f_s = symbols("f_s") #scale factor.
        f_s_val = 63.5 #mm
```

[25]:
```
        #define m in terms of r
        m_r = f_s*r + Rot*delta

        #display
        Eqdisp("m_r",m_r)
        Eqdisp("f_s",f_s_val)
```

$$m_r = \begin{bmatrix} \Delta_1\cos\left(\theta\right)-\Delta_2\sin\left(\theta\right)+f_s r_1 \\ \Delta_1\sin\left(\theta\right)+\Delta_2\cos\left(\theta\right)+f_s r_2 \end{bmatrix}$$

$f_s = 63.5$

[26]:
```
        #define 3-vectors to act as full configurations.
        mx = sym.Matrix([m1,m2,th])
        rx = sym.Matrix([r1,r2,th])
        px = sym.Matrix([p1,p2,th])

        #make substitutions to make it differentiable
        mxsub = mx.subs({m1:m_r[0],m2:m_r[1]})
        rxsub = rx.subs({r1:r_p[0],r2:r_p[1]}).subs({w1:w_x[0],w2:w_x[1]})

        #calculate jacobians for velocity kinematics
        dm_dr = mxsub.jacobian(rx)
        dr_dp = rxsub.jacobian(px)

        #getting dm_dp
        dm_dp = dm_dr*dr_dp

        #display
        Eqdisp('m_{rx}',mxsub,label='m and r as a funcion of r and p,␣
↪respectively. Theta as the third coordinate')
        Eqdisp('r_{px}',rxsub)
       ␣
↪Eqdisp(r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{t}}',r'\frac{\mathrm{d}\mathbf{m}}{\ma
            label='conversion from dr_dt to dm_dt')
        Eqdisp(r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}}',dm_dr)
       ␣
↪Eqdisp(r'\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{t}}',r'\frac{\mathrm{d}\mathbf{r}}{\ma
            label='conversion from dp_dt to dr_dt')
        Eqdisp(r'\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{p}}',dr_dp)
        Eqdisp(r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{t}}',
              ␣
↪r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}}\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mat
            label='conversion from dp_dt to dm_dt')
        Eqdisp(r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{p}}',
              ␣
↪r'\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}}\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mat
            dm_dp)
```

m and r as a funcion of r and p, respectively. Theta as the third␣
↪coordinate

$$m_{rx} = \begin{bmatrix} \Delta_1 \cos(\theta) - \Delta_2 \sin(\theta) + f_s r_1 \\ \Delta_1 \sin(\theta) + \Delta_2 \cos(\theta) + f_s r_2 \\ \theta \end{bmatrix}$$

$$r_{px} = \begin{bmatrix} p_1 (-s \cos(\theta) + 1) + p_2 s \sin(\theta) \\ -p_1 s \sin(\theta) + p_2 (-s \cos(\theta) + 1) \\ \theta \end{bmatrix}$$

```
conversion from dr_dt to dm_dt
```

$$\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{t}} = \frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}} \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{t}}$$

$$\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}} = \begin{bmatrix} f_s & 0 & -\Delta_1 \sin(\theta) - \Delta_2 \cos(\theta) \\ 0 & f_s & \Delta_1 \cos(\theta) - \Delta_2 \sin(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

```
conversion from dp_dt to dr_dt
```

$$\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{t}} = \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{p}} \frac{\mathrm{d}\mathbf{p}}{\mathrm{d}\mathbf{t}}$$

$$\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{p}} = \begin{bmatrix} -s \cos(\theta) + 1 & s \sin(\theta) & p_1 s \sin(\theta) + p_2 s \cos(\theta) \\ -s \sin(\theta) & -s \cos(\theta) + 1 & -p_1 s \cos(\theta) + p_2 s \sin(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

```
conversion from dp_dt to dm_dt
```

$$\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{t}} = \frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}} \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{p}} \frac{\mathrm{d}\mathbf{p}}{\mathrm{d}\mathbf{t}}$$

$$\frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{p}} = \frac{\mathrm{d}\mathbf{m}}{\mathrm{d}\mathbf{r}} \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}\mathbf{p}} = \begin{bmatrix} f_s (-s \cos(\theta) + 1) & f_s s \sin(\theta) & -\Delta_1 \sin(\theta) - \Delta_2 \cos(\theta) + f_s (p_1 s \sin(\theta) + p_2 s \cos(\theta)) \\ -f_s s \sin(\theta) & f_s (-s \cos(\theta) + 1) & \Delta_1 \cos(\theta) - \Delta_2 \sin(\theta) + f_s (-p_1 s \cos(\theta) + p_2 s \sin(\theta)) \\ 0 & 0 & 1 \end{bmatrix}$$

3.11.2 "backwards" conversion from m to r and p

[27]:
```
#lets get r in terms of m...
r_m = (1/f_s)*(m-Rot*delta)

#lets get p in terms of m...
p_m = simplify(p_r.subs({r1:r_m[0],r2:r_m[1]}))
# p_m

#displays
Eqdisp('r1_m',r_m[0],label='r in terms of m')
Eqdisp('r2_m',r_m[1])
Eqdisp('p_r',p_r,label='recall, p in terms of r')
Eqdisp('p1_m',p_m[0],label='substituted in -> p in terms of m')
Eqdisp('p2_m',p_m[1])
```

r in terms of m

$$r_{1m} = \frac{-\Delta_1 \cos(\theta) + \Delta_2 \sin(\theta) + m_1}{f_s}$$

$$r_{2m} = \frac{-\Delta_1 \sin(\theta) - \Delta_2 \cos(\theta) + m_2}{f_s}$$

recall, p in terms of r

$$p_r = \begin{bmatrix} \frac{r_1 w_2}{w_1^2+w_2^2} - \frac{r_2 w_1}{w_1^2+w_2^2} \\ \frac{r_1 w_1}{w_1^2+w_2^2} + \frac{r_2 w_2}{w_1^2+w_2^2} \end{bmatrix}$$

substituted in -> p in terms of m

$$p_{1m} = \frac{w_1 (\Delta_1 \sin(\theta) + \Delta_2 \cos(\theta) - m_2) + w_2 (-\Delta_1 \cos(\theta) + \Delta_2 \sin(\theta) + m_1)}{f_s (w_1^2 + w_2^2)}$$

$$p_{2m} = \frac{w_1 (-\Delta_1 \cos(\theta) + \Delta_2 \sin(\theta) + m_1) - w_2 (\Delta_1 \sin(\theta) + \Delta_2 \cos(\theta) - m_2)}{f_s (w_1^2 + w_2^2)}$$